Understanding Supply Chain Criticality Prediction using Graph Neural Networks

Graph Based Criticality Prediction in Supply Chain Networks

Connor McNeil
Mathematics & Computing Science
Saint Mary's University
Halifax, Nova Scotia, Canada
Connor.McNeil@smu.ca

Yilin Huang
Mathematics & Computing Science
Saint Mary's University
Halifax, Nova Scotia, Canada
Yilin.Huang@smu.ca

Connor Gerrard
Mathematics & Computing Science
Saint Mary's University
Halifax, Nova Scotia, Canada
Connor.Gerrard@smu.ca

ABSTRACT

Supply chains are complex systems where many different entities depend on each other. If one important node fails, it can affect a large part of the network. Because of this, identifying critical nodes is important for improving reliability and planning backups.

In this project, a supply chain is modeled as a graph where nodes represent entities such as ports, plants, warehouses, and distribution centers, and edges represent the movement of goods. We compare traditional machine learning models with a Graph Neural Network (GraphSAGE) to see if using graph structure improves performance.

The results show that GraphSAGE performs better overall, especially in identifying critical nodes. This is mainly due to its ability to use information from neighboring nodes, allowing it to detect risks that are not visible from node features alone. However, traditional models such as Random Forest still perform well when strong feature patterns exist, particularly in maintaining higher precision.

To address this, a final model was developed using a stratified 5-fold cross-validated ensemble combining Random Forest (60%) and GraphSAGE (40%). This approach achieved the best overall performance by balancing precision and recall, reducing false positives while still capturing critical nodes. These results show that using both feature-based and graph-based learning provides a more complete and reliable approach for analyzing supply chain networks.

KEYWORDS

Graph Neural Networks; GraphSAGE; Supply Chain Resilience; Node Classification; Ensemble Learning; PyTorch Geometric; Infrastructure Criticality; Feature Engineering; 5-Fold Cross-Validation.

ACM Reference format:

Connor Gerrard, Connor McNeil, Yilin Huang. 2026 .Supply Chain Criticality Prediction using Graph Neural Networks: Demonstration of Practical ML in Infrastructure Analysis. In *Proceedings of ACM Woodstock conference (WOODSTOCK'18). ACM, New York, NY, USA, 2 pages.* https://doi.org/10.1145/1234567890

1 INTRODUCTION

1.1 BACKGROUND

Modern supply chains are not simple or linear systems. Instead, they are large networks made up of many interconnected entities. These entities include ports, plants, warehouses, and distribution centers, all working together to move goods through different stages of production and distribution. Because of this level of connectivity, failures at certain nodes can have a large impact on the rest of the system.

In many cases, a disruption at a single critical node can affect multiple downstream processes. This can lead to delays, shortages, or even complete breakdowns in parts of the supply chain. As a result, identifying which nodes are critical is an important problem for improving resilience and planning backup strategies.

1.2 PROBLEM STATEMENT

The main challenge in this problem is that node importance is not determined only by local features. A node might appear unimportant based on its own attributes, but still be critical because of how it connects different parts of the network. Traditional machine learning models do not capture this type of relationship, since they treat each node independently.

This project addresses that limitation by modeling the supply chain as a graph and applying Graph Neural Networks. The goal is to determine whether using graph structure improves the ability to classify nodes as critical or non-critical.

1.3 OBJECTIVES

The objectives of this project are to:

- Build a graph representation of a supply chain network
- Engineer meaningful features for each node
- Compare traditional machine learning models with a graph-based model
- Evaluate performance using both single splits and cross-validation
- Develop a final model that balances precision and recall

2 METHODOLOGY

2.1 DATASET AND GRAPH CONSTRUCTION

The supply chain is represented as a directed graph consisting of approximately 600 nodes and 1,593 edges. The nodes include 400 infrastructure nodes, which are the primary focus of the classification task, and 200 additional context nodes representing products and ingredients. These context nodes do not have labels, but they play an important role in defining the structure of the graph.

The labeled infrastructure set consists of 175 distribution centers, 90 plants, 120 warehouses, and 15 ports.

Edges represent relationships between nodes and capture how goods move through the supply chain. This allows the graph to represent both direct and indirect dependencies between entities.

The labeled portion of the dataset consists of 240 critical nodes and 160 non-critical nodes. This creates a moderate class imbalance, which needs to be considered during training. The presence of unlabeled nodes also reflects a more realistic scenario, where not all parts of a system are fully annotated but still influence the overall structure.

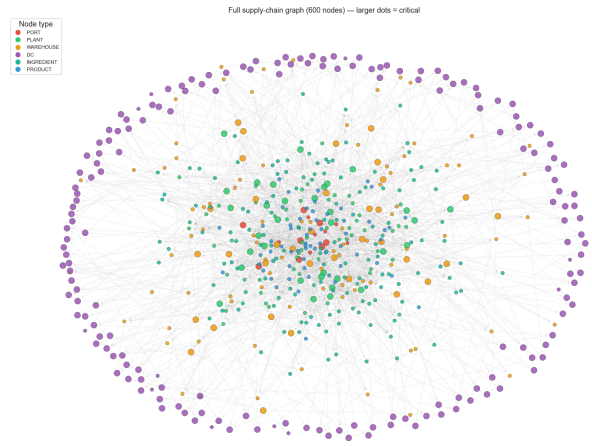

Figure 1 Graph Visualization

2.2 FEATURE ENGINEERING

Each node is represented using a vector of 33 features. These features are designed to capture both the properties of individual nodes and their role within the network.

The features include node type encodings, which indicate whether a node is a port, plant, warehouse, or distribution center. In addition, structural features such as degree and connectivity are included to give baseline models some information about graph structure. Other features capture operational aspects of the supply chain, such as capacity or recovery time.

Although these features provide useful information, they are not sufficient on their own to fully describe node importance. This is one of the reasons why a graph-based approach is necessary.

2.3 GRAPHSAGE MODEL

The main model used in this project is a 2-layer GraphSAGE Graph Neural Network. GraphSAGE works by updating node representations through aggregation of neighbor information. At each layer, a node combines its

own features with the features of its neighbors, allowing it to learn patterns based on the local structure of the graph.

The model was implemented using PyTorch Geometric. After tuning, the final configuration used a hidden dimension of 128, a dropout rate of 0.3, and a learning rate of 0.01. The model converged relatively quickly, with the best validation performance reached around epoch 35.

This approach allows the model to capture relationships that are not visible when looking at nodes individually.

2.4 BASELINE MODELS

To provide a comparison, two traditional machine learning models were used.

Logistic Regression was included as a simple baseline. It is a linear model that relies entirely on the input features. While it is limited in complexity, it provides a useful reference point for how well the features perform on their own.

Random Forest was also used as a stronger baseline. It is capable of capturing non-linear relationships between features and generally performs better than Logistic Regression in more complex settings. However, like Logistic Regression, it does not incorporate graph structure.

2.5 TRAINING & EVALUATIONS STRATEGY

The project uses two evaluation approaches. The first is a standard train, validation, and test split, which was used for initial experiments and model tuning. The labeled nodes were split into 280 training samples, 60 validation samples, and 60 test samples.

The second approach is the final evaluation, which uses stratified 5-fold cross-validation on the 400 labeled infrastructure nodes. This approach provides a more reliable estimate of model performance, especially given the relatively small dataset and class imbalance.

During cross-validation, models are trained and evaluated across multiple splits, and performance is averaged across folds. This reduces the impact of any single split and gives a more stable result.

3 RESULTS AND ANALYSIS

3.1 EXPLORATORY DATA ANALYSIS

The exploratory analysis revealed several important patterns in the dataset. Distribution centers were found to be highly likely to be critical, while plants and warehouses were more likely to be non-critical. Ports had a more balanced distribution, which made them harder to classify. As shown in Figure 2, distribution centers are mostly labeled as critical.

In addition, the continuous features showed significant overlap between critical and non-critical nodes. As shown in Figure 3, the feature distributions overlap considerably. This means that no single feature is sufficient to separate the classes. Instead, the classification task requires combining multiple features and, importantly, using structural information from the graph. As shown in Figure 4, some features are more strongly related to criticality than others.

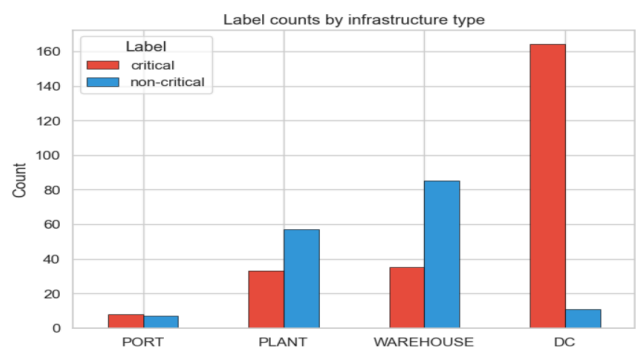

Figure 2 Node labels by infrastructure type.

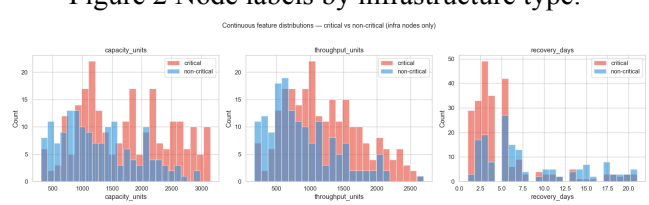

Figure 3 Feature distributions by label.

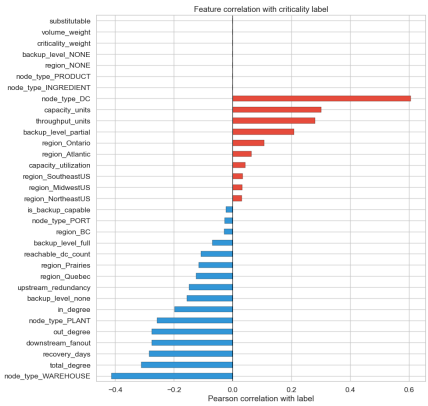

Figure 4 Feature correlation with label.

3.2 BASELINE MODEL RESULTS

The baseline models performed reasonably well using only node features. Logistic Regression achieved high precision, meaning it was accurate when predicting critical nodes. However, it had lower recall, which means it missed some critical nodes.

Random Forest improved on this by capturing more complex relationships between features. As shown in Figure 5, the models differ mainly in how they balance false positives and false negatives. It achieved a better balance between precision and recall, but still had limitations due to the lack of graph information.

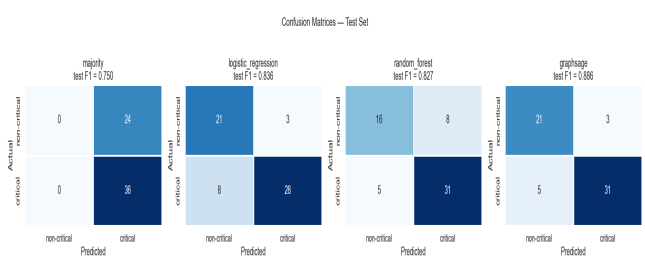

Figure 5 Confusion matrices for all models.

3.3 GRAPHSAGE RESULTS

The GraphSAGE model showed a different pattern of performance. It achieved higher recall than the baseline models, meaning it was better at identifying critical nodes. This is likely due to its ability to use information from neighboring nodes.

However, this also led to lower precision, as the model sometimes classified non-critical nodes as critical. This behavior reflects the model's sensitivity to structural patterns in the graph.

Overall, GraphSAGE achieved the best performance among individual models, particularly in terms of F1 score and ROC-AUC. As shown in Figure 6, the model converges steadily and reaches its best performance during training.

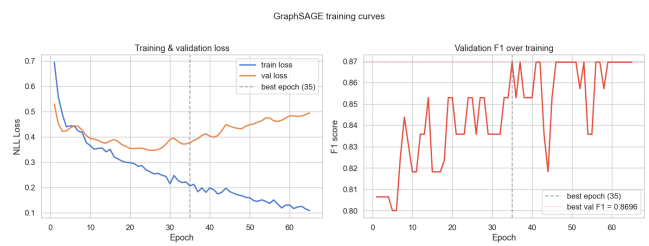

Figure 6 GraphSAGE training curves (loss + validation F1)

3.4 ENSEMBLE RESULTS

To address the tradeoff between precision and recall, a weighted ensemble model was developed. This model combines predictions from Random Forest and GraphSAGE using a weighted average, with 60% weight assigned to Random Forest and 40% to GraphSAGE. As shown in Figure 7, this weighting gives the strongest overall performance.

The ensemble was evaluated using stratified 5-fold cross-validation and achieved the best overall performance. As shown in Figure 8, the ensemble performs best overall across the main evaluation metrics. By combining the strengths of both models, it reduced false positives while still maintaining strong detection of critical nodes.

The improvement comes from the fact that the two models make different types of errors. Random Forest is strong when patterns exist in the features, while GraphSAGE is strong when patterns exist in the graph structure. Combining them results in more balanced and stable predictions.

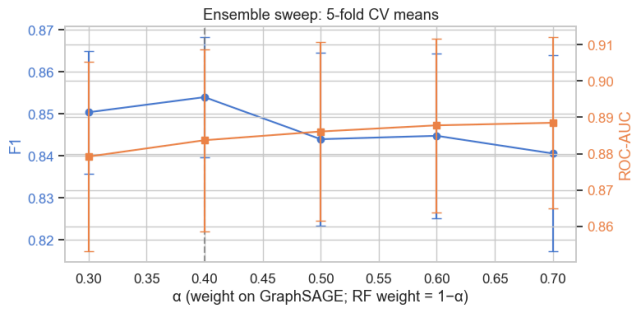

Figure 7 Ensemble weight tuning

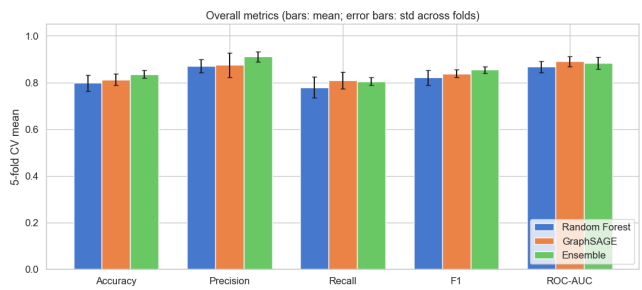

Figure 8 Model performance comparison.

3.5 SUMMARY OF RESULTS

The results show that GraphSAGE is the best individual model due to its ability to capture graph structure. However, the ensemble model provides the best overall performance by balancing precision and recall.

This confirms that both feature-based and graph-based information are important for this problem.

DISCUSSION

The main takeaway from this project is that supply chain criticality is not purely a feature-based problem. Instead, it is strongly influenced by the structure of the network.

Traditional models perform well when feature patterns are clear, but they fail to capture relationships between nodes. GraphSAGE improves this by incorporating neighbor information, allowing it to identify nodes that are important due to their position in the graph.

However, GraphSAGE alone is not perfect. Its tendency to produce false positives shows that structural information can sometimes introduce noise. This is why the ensemble model performs best, as it combines the strengths of both approaches.

Another important point is the use of cross-validation. By evaluating models across multiple splits, the results become more reliable and less dependent on a single dataset partition.

CONCLUSIONS

This project showed that Graph Neural Networks are effective tools for analyzing supply chain networks. The 2-layer GraphSAGE model improves the ability to identify critical nodes by using graph structure, while traditional models provide strong performance based on features.

The final ensemble model, combining Random Forest and GraphSAGE, achieves the best overall results. This shows that combining different types of models can lead to better performance than using a single approach.

Overall, the project highlights the importance of using both feature-based and graph-based learning when working with complex systems.

RECOMMENDATIONS

Future work could explore additional Graph Neural Network architectures, such as Graph Attention Networks, to further improve performance. It would also be useful to test the model on larger or real-world supply chain datasets.

Another direction would be to study dynamic graphs, where connections between nodes change over time. This would provide a more realistic representation of real-world supply chains.

Finally, further improvements could be made to the ensemble approach by testing different weighting strategies or combining additional models.

REFERENCES

[1] Neo4j, Inc. (n.d.). *Neo4j Graph Database Documentation*. https://neo4j.com/docs/

[2] PyTorch Geometric Team. (n.d.). *PyTorch Geometric Documentation*. https://pytorch-geometric.readthedocs.io/

[3] PyTorch. (n.d.). *PyTorch Documentation*. https://pytorch.org/docs/

[4] Scikit-learn. (n.d.). *Scikit-learn Documentation*. https://scikit-learn.org/stable/documentation.html

[5] NumPy. (n.d.). *NumPy Documentation*. https://numpy.org/doc/

[6] Pandas. (n.d.). *Pandas Documentation*. https://pandas.pydata.org/docs/

[7] Matplotlib. (n.d.). *Matplotlib Documentation*. https://matplotlib.org/stable/contents.html

[8] NetworkX. (n.d.). *NetworkX Documentation*. https://networkx.org/documentation/stable/